Universidade Federal do Pará
Instituto de Tecnologia (ITEC)
Faculdade de Engenharia da Computação e Telecomunicações (FCT)

Inteligência Computacional: Exercícios Resolvidos de Redes Neurais Convolucionais

1) Dada a rede a seguir, construída a partir do framework `Tensorflow`, entenda e justifique as dimensões indicadas (à direita no campo `# param`) para os tensores na saída de cada camada da rede abaixo. Encontre uma fórmula para calcular as dimensões de saída para as camadas convolucional e densa.

```
model = models.sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Flatten())
model.add(Dense(units=512, activation='relu'))
model.add(Dense(units=1, activation='simgmoid'))
model.summary()
```

A saída da linha `model.summary()` expressa essas informações:

Tabela 1: Descrição em termos de dimensão do modelo construído.

Layer (type)	**Output shape**	**Param #**
conv2d_1 (Conv2D)	(None, 148, 148, 32)	$896 = ((3 \times 3) \times 3 + 1) \times 32)$
max_pooling2d_1 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_2 (Conv2D)	(None, 72, 72, 64)	$18496 = ((3 \times 3) \times 32 + 1) \times 64)$
max_pooling2d_2 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_3 (Conv2D)	(None, 34, 34, 128)	$73856 = ((3 \times 3) \times 64 + 1) \times 128)$
max_pooling2d_3 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_4 (Conv2D)	(None, 15, 15, 128)	$147584 = ((3 \times 3) \times 128 + 1) \times 128)$
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten_1 (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 512)	$3211776 = (6272 + 1) \times 512$
dense_2 (Dense)	(None, 1)	513

2) A imagem de entrada $\mathbf{X}$ com 4×4 pixels de uma rede convolucional simples é monocromática, possuindo apenas um canal e sendo dada por

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 3 & 0 \\ 250 & 2 & 0 & 0 \\ 3 & 1 & 255 & 1 \\ 1 & 0 & 3 & 2 \end{bmatrix}.$$

A primeira camada escondida é convolucional e usa dois filtros, dados por

$$\mathbf{F}_{1,1,1} = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix},$$

$$\mathbf{F}_{1,2,1} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}.$$

A nomenclatura do filtro $\mathbf{F}_{c,f,d}$ denota camada c, filtro f e dimensão da profundidade. Note que neste caso o "depth" da entrada $\mathbf{X}$ é D=1, e daí cada um dos dois filtros também tem `depth` = 1. A convolução é feita com stride = 1 e com `padding` (`padding`='same' na convenção do `Keras`), de forma que a saída $\mathbf{Y}$ de cada convolução tenha a mesma dimensão de $\mathbf{X}$.

a) Qual a dimensão e valores do tensor de saída desta primeira camada escondida considerando-se que a função de ativação é a ReLU? **b)** Considerando que a segunda camada desta rede é convolucional também, mas utiliza os kernels abaixo

$$\mathbf{F}_{2,1,1} = \begin{bmatrix} 2 & -1 \\ 1 & 0 \end{bmatrix},$$

$$\mathbf{F}_{2,1,2} = \begin{bmatrix} -2 & 1 \\ 0 & -1 \end{bmatrix},$$

stride = 2, `ReLU` e não usa `padding` (no `Keras`: `padding` = valid), qual a dimensão e valores do tensor de saída desta segunda camada escondida quando a entrada é a mesma da calculada no item **a)**? Todas estas camadas convolucionais não utilizam o bias.

Resolução

Como descrito anteriormente como "detalhe", como a dimensão do kernel é par, o `Tensorflow` e `Keras` acrescentariam uma coluna de zeros à direita de $\mathbf{X}$ e uma linha de zeros abaixo. Seguiremos esta convenção. Portanto, a nova matriz com padding é:

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 3 & 0 & 0 \\ 250 & 2 & 0 & 0 & 0 \\ 3 & 1 & 255 & 1 & 0 \\ 1 & 0 & 3 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Após o padding, assume-se que se deseja na saída apenas as parcelas da convolução com regiões "válidas" da entrada, e se tem para o filtro $\mathbf{F}_{1,1,1}$, a seguinte saída

$$\mathbf{Y}_1 = \begin{bmatrix} 249 & -1 & 3 & 0 \\ 250 & -252 & 254 & 1 \\ 3 & -257 & 255 & 3 \\ 1 & -3 & 1 & 2 \end{bmatrix},$$

e similarmente, para $\mathbf{F}_{1,2,1}$:

$$\mathbf{Y}_2 = \begin{bmatrix} -251 & 1 & 3 & 0 \\ 248 & -254 & -256 & -1 \\ 3 & 253 & 251 & -1 \\ 1 & 3 & 5 & 2 \end{bmatrix}.$$

Após $\mathbf{Y}_1$ e $\mathbf{Y}_2$ passarem pela função `ReLU` $f(\mathbf{X})$, tem-se:

$$\mathbf{H}_1 = \begin{bmatrix} 249 & 0 & 3 & 0 \\ 248 & 0 & 254 & 1 \\ 3 & 0 & 255 & 3 \\ 1 & 0 & 1 & 2 \end{bmatrix},$$

e

$$\mathbf{H}_2 = \begin{bmatrix} 0 & 1 & 3 & 0 \\ 250 & 0 & 0 & 0 \\ 3 & 253 & 251 & 0 \\ 1 & 3 & 5 & 2 \end{bmatrix}.$$

$\mathbf{H}_1$ e $\mathbf{H}_2$ são as entradas para a segunda camada, que tem então uma entrada com depth=2 (note que o depth da entrada na primeira camada era 1). O filtro 2-D da segunda camada é composto $\mathbf{F}_{2,1,1}$ por $\mathbf{F}_{2,1,2}$ e processa cada uma das duas dimensões usando stride=2 e `padding='valid'` e gera para $\mathbf{H}_1$:

$$\mathbf{P}_1 = \begin{bmatrix} 748 & 260 \\ 7 & 508 \end{bmatrix},$$

e para $\mathbf{H}_2$

$$\mathbf{P}_2 = \begin{bmatrix} 1 & -6 \\ 244 & -504 \end{bmatrix}.$$

com estas parcelas sendo somadas para obter a "net input" do neurônio na 2a camada:

$$\mathbf{P} = \mathbf{P}_1 + \mathbf{P}_2 = \begin{bmatrix} 749 & 254 \\ 251 & 4 \end{bmatrix}.$$

E após passar pela função de ativação `ReLU`:

$$\mathbf{P} = \mathbf{P}_1 + \mathbf{P}_2 = \begin{bmatrix} 749 & 254 \\ 251 & 4 \end{bmatrix}.$$

3) Qual a saída de uma camada `MaxPooling2D` (por exemplo, a do `Keras`) criada com a instrução `layers.MaxPooling2D((2, 2))`, quando sua entrada é um tensor de dimensão 4×4 dado por

$$\mathbf{X} = \begin{bmatrix} 1 & 140 & 3 & 0 \\ 250 & 2 & 34 & -2 \\ 3 & 1 & -255 & 1 \\ 1 & -23 & 3 & 2 \end{bmatrix}.$$

Como não é especificado, o stride (por "default") é 2 e não há `padding` (`padding` = valid).

Resolução
O `layers.MaxPooling2D((2, 2))` é o *max pooling* mais usual, e leva em consideração o valor máximo de um dado conjunto. Nesse sentido, como essa camada foi declarada com dimensão de 2×2, a saída calculada leva em consideração as regiões hachuradas para cada cor.

$$\mathbf{X} = \begin{bmatrix} 1 & 140 & 3 & 0 \\ 250 & 2 & 34 & -2 \\ 3 & 1 & -255 & 1 \\ 1 & -23 & 3 & 2 \end{bmatrix}.$$

Portanto, a saída final $\mathbf{Y}$ é:

$$\mathbf{Y} = \begin{bmatrix} 250 & 34 \\ 3 & 3 \end{bmatrix}.$$

4) Uma camada MaxPooling2D é inicializada no Keras com a instrução:

```
layers.MaxPooling2D((5,5), padding='same', strides=(2, 2)).
```

O tensor $\mathbf{Z}$ em sua entrada tem dimensão 4×4 e é dado por:

$$\mathbf{Z} = \begin{bmatrix} 1 & 0 & 3 & 2 \\ -5 & 2 & 0 & 4 \\ 3 & 8 & 5 & 0 \\ 1 & 0 & 0 & -2 \end{bmatrix}.$$

Qual o tensor de saída desta camada (informe a dimensão do tensor e o valor de seus elementos)?
Resolução:
Para resolver essa questão, é necessário saber que, independente do valor do `stride`, atributo `padding='same'` irá adicionar $F-1$ linhas e colunas, em que F é a dimensão do kernel da camada de `MaxPooling2D`, $F = 5$. Portanto, após fazer o padding, temos o seguinte tensor:

$$\mathbf{Z}_{\text{padded}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 3 & 2 & 0 & 0 \\ 0 & 0 & -5 & 2 & 0 & 4 & 0 & 0 \\ 0 & 0 & 3 & 8 & 5 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & -2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Considerando um `stride` 2×2, temos, portanto, a saída final sendo um tensor com dimensões 2×2:

$$\mathbf{Z}_{\text{out}} = \begin{bmatrix} 8 & 8 \\ 8 & 8 \end{bmatrix}$$

5) O tensor de entrada de uma rede convolucional possui dimensão $4 \times 4 \times 2$, onde 2 é o "depth" (profundidade). Este tensor é descrito pelas matrizes

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 2 & 0 & 0 & 1 \end{bmatrix}.$$

e

$$\mathbf{X}_2 = \begin{bmatrix} 2 & 0 & 2 & 2 \\ -2 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}.$$

A primeira camada escondida é convolucional, com função de ativação ReLU, e usa dois filtros para a convolução K-D. As convoluções são feitas com `strides` $= (1, 2)$ e sem "zero padding" (`padding='valid'` na convenção do Keras). O primeiro filtro é composto pelos kernels

$$\mathbf{F}_1 = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix},$$

$$\mathbf{F}_2 = \begin{bmatrix} -1 & 1 \\ 1 & 0 \end{bmatrix},$$

com peso do "bias" sendo 1.5.
O segundo filtro é composto pelos kernels

$$\mathbf{G}_1 = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix},$$

e

$$\mathbf{G}_2 = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix},$$

com o peso do "bias" sendo 3.
(a) Qual o número de parâmetros desta camada que podem ser treinados? **(b)** Qual é o tensor de saída desta primeira camada (informe a dimensão do tensor e o valor de seus elementos)?

Resolução:
(a) Podem ser treinados os elementos de cada kernel, ou seja, 16 parâmetros mais os pesos dos "bias", totalizando 18 parâmetros.
(b) O "bias" é aplicado à matriz obtida pela soma das convoluções com todos os kernels.
Para o 1º filtro:

$$\mathbf{Z}_1 = \mathbf{X}_1 * \mathbf{F}_1 + \mathbf{X}_2 * \mathbf{F}_2$$

Portanto, realizando as convoluções, temos que:

$$\mathbf{Z}_1 = \begin{bmatrix} 2 & 1 \\ 0 & 0 \\ 0 & 2 \end{bmatrix} + \begin{bmatrix} -4 & 0 \\ 4 & 0 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} -2 & 1 \\ 4 & 0 \\ 1 & 2 \end{bmatrix}$$

Somando a $\mathbf{Z}_1$ o "bias" 1 multiplicado pelo peso 1.5, temos que a saída final é:

$$\mathbf{Y}_1 = \text{ReLU}\left\{\mathbf{Z}_1 + \begin{bmatrix} 1.5 \\ 1.5 \\ 1.5 \end{bmatrix}\right\} = \text{ReLU}\left\{\begin{bmatrix} -0.5 & 2.5 \\ 5.5 & 1.5 \\ 2.5 & 3.5 \end{bmatrix}\right\} = \begin{bmatrix} 0 & 2.5 \\ 5.5 & 1.5 \\ 2.5 & 3.5 \end{bmatrix}.$$

Fazendo um processo similar para o segundo filtro, temos que a saída é igual a:

$$\mathbf{Y}_2 = \begin{bmatrix} 10 & 7 \\ 2 & 4 \\ 7 & 8 \end{bmatrix}.$$